

DBS302 - NoSQL Database Management

Practical 1A: Leaderboard with Redis Sorted Sets

Student Name: Tandin Om

Student Number: 02230302

1. Aim

To design and implement a real-time leaderboard system using Redis data structures, specifically sorted sets, to efficiently store and retrieve player ranking.

2. Objectives

- To discuss the nature and application of Redis data structures, namely sorted sets (ZSET)
- To design and implement a leaderboard system using ZADD, ZREVRANGE, ZREVRANK, and ZINCRBY commands
- To design and implement a Python class encapsulating operations to manipulate leaderboards
- To design and implement pagination and "around me" queries
- To complete exercises on daily and all-time leaderboards, score ranges, and time to live
- The Redis server was started, and a connection was established to the Redis server using a Python script. The ping command was issued, and ping() returned True, indicating a successful connection to Redis. A test key-value pair was added and retrieved from Redis.

3. Common Setup - Redis Connectivity Test

```
● (venv) macbookairm4chip@Tandin-0mu redis-practicals % python test_redis_connection.py
Redis PING response: True
Value from Redis: hello-redis
❏ (venv) macbookairm4chip@Tandin-0mu redis-practicals %
```

The Redis server was started and the connectivity was checked by executing a Python script. The ping() command was executed successfully by returning True for the connection. The key-value pair was also stored and retrieved successfully.

4. Practical 1A - Leaderboard Implementation

4.1 Redis CLI Demonstration

```
(venv) macbookairm4chip@Tandin-0mu redis-practicals % redis-cli
127.0.0.1:6379> ZADD leaderboard:game:season1 1500 alice
(integer) 1
127.0.0.1:6379> ZADD leaderboard:game:season1 2300 bob
(integer) 1
127.0.0.1:6379> ZADD leaderboard:game:season1 1800 charlie
(integer) 1
127.0.0.1:6379> ZADD leaderboard:game:season1 2100 diana
(integer) 1
127.0.0.1:6379> ZADD leaderboard:game:season1 1950 eve
(integer) 1
127.0.0.1:6379> ZREVRANGE leaderboard:game:season1 0 2 WITHSCORES
1) "bob"
2) "2300"
3) "diana"
4) "2100"
5) "eve"
6) "1950"
127.0.0.1:6379> ZREVRANK leaderboard:game:season1 charlie
(integer) 3
127.0.0.1:6379> ZSCORE leaderboard:game:season1 charlie
"1800"
127.0.0.1:6379> ZINCRBY leaderboard:game:season1 500 charlie
"2300"
127.0.0.1:6379> ZREVRANGE leaderboard:game:season1 0 2 WITHSCORES
1) "charlie"
2) "2300"
3) "bob"
4) "2300"
5) "diana"
6) "2100"
127.0.0.1:6379> █
```

The following Redis CLI commands were executed to demonstrate leaderboard operations:

Command	Purpose
ZADD leaderboard:game:season1 1950 eve	Add player with score
ZREVRANGE ... 0 2 WITHSCORES	Get top 3 players

ZREVRANK ... charlie	Get Charlie's rank (returns 3, 0-based)
ZSCORE ... charlie	Get Charlie's score (1800)
ZINCRBY ... 500 charlie	Increment Charlie's score by 500
ZREVRANGE ... 0 2 WITHSCORES	View updated top 3 (Charlie now #1)

4.2 Python Implementation

```
(venv) macbookairm4chip@Tandin-0mu redis-practicals % python leaderboard.py
Adding initial scores...

Top 3 players:
#1 bob: 2300.0
#2 diana: 2100.0
#3 eve: 1950.0

Charlie's current rank and score:
Rank: 4
Score: 1800.0

Incrementing Charlie's score by 500...
New score: 2300.0, new rank: 1

Players around Charlie:
#1 charlie: 2300.0
#2 bob: 2300.0
#3 diana: 2100.0

Page 1 of leaderboard (page_size=3):
#1 charlie: 2300.0
#2 bob: 2300.0
#3 diana: 2100.0
(venv) macbookairm4chip@Tandin-0mu redis-practicals %
```

A leaderboard class was implemented in Python with the following methods:

Method	Description
add_score()	Add or update player score
increment_score()	Increase player score
get_rank()	Get player's 1-based rank

<code>get_score()</code>	Get player's current score
<code>get_top()</code>	Get top N players
<code>get_page()</code>	Get paginated results
<code>get_around_player()</code>	Get players around a specific player

Output Summary:

- Initial top 3: Bob (2300), Diana (2100), Eve (1950)
- Charlie was ranked 4th with 1800 points
- Charlie moved to rank 1 with 2300 points after +500 increment
- Pagination and "around player" features worked correctly

5. Exercises

Exercise 1: Daily and All-Time Leaderboards

```
(env) macbookairm4chip@Tandin-0mu redis-practicals % redis-cli
127.0.0.1:6379> ZADD leaderboard:game:daily:2026-03-18 1200 alice
(integer) 0
127.0.0.1:6379> ZADD leaderboard:game:daily:2026-03-18 2000 bob
(integer) 0
127.0.0.1:6379> ZADD leaderboard:game:alltime 8000 alice
(integer) 0
127.0.0.1:6379> ZADD leaderboard:game:alltime 9500 bob
(integer) 0
127.0.0.1:6379> ZREVRANGE leaderboard:game:daily:2026-03-18 0 2 WITHSCORES
1) "charlie"
2) "2300"
3) "diana"
4) "2100"
5) "bob"
6) "2000"
127.0.0.1:6379> ZREVRANGE leaderboard:game:alltime 0 2 WITHSCORES
1) "bob"
2) "9500"
3) "alice"
4) "8000"
5) "charlie"
6) "2300"
127.0.0.1:6379> ZREVRANGEBYSCORE leaderboard:game:daily:2026-03-18 3000 1000 WITHSCORES
1) "charlie"
2) "2300"
3) "diana"
4) "2100"
5) "bob"
6) "2000"
7) "eve"
8) "1950"
9) "alice"
10) "1200"
127.0.0.1:6379> EXPIRE leaderboard:game:daily:2026-03-18 604800
(integer) 1
127.0.0.1:6379> TTL leaderboard:game:daily:2026-03-18
(integer) 604791
127.0.0.1:6379> █
```

```

(venv) macbookairm4chip@Tandin-0mu redis-practicals % python leaderboard.py

Adding initial scores...

Top 3 players:
#1 bob: 2300.0
#2 diana: 2100.0
#3 eve: 1950.0

Charlie's current rank and score:
Rank: 4
Score: 1800.0

Incrementing Charlie's score by 500...
New score: 2300.0, new rank: 1

Players around Charlie:
#1 charlie: 2300.0
#2 bob: 2300.0
#3 diana: 2100.0

Page 1 of leaderboard (page_size=3):
#1 charlie: 2300.0
#2 bob: 2300.0
#3 diana: 2100.0

--- Exercise 1: Daily and All-Time Leaderboards ---

Daily Leaderboard Top 3:
#1 charlie: 2300.0
#2 bob: 2300.0
#3 diana: 2100.0

All-Time Leaderboard Top 3:
#1 charlie: 2300.0
#2 bob: 2300.0
#3 diana: 2100.0

--- Exercise 2: Players with scores between 1800 and 2200 ---
diana: 2100.0
eve: 1950.0

--- Exercise 3: Setting TTL of 7 days on daily leaderboard ---
TTL set on 'leaderboard:game:daily:2026-03-17': 604800 seconds (~7 days = 604800 seconds)
(venv) macbookairm4chip@Tandin-0mu redis-practicals %

```

Two separate leaderboard keys were created:

- `leaderboard:game:daily:2026-03-18` - tracks daily scores
- `leaderboard:game:alltime` - tracks all-time scores

Both leaderboards successfully maintained independent score tracking.

Exercise 2: Score Range Query

Using `ZREVRANGEBYSCORE`, players with scores between 1000 and 3000 were retrieved:

Player	Score
Charlie	2300
Diana	2100
Bob	2000
Eve	1950
Alice	1200

Exercise 3: TTL on Daily Leaderboard

An expiration time of 7 days (604,800 seconds) was set on the daily leaderboard using `EXPIRE`. The `TTL` command confirmed the expiration was set correctly:

- `EXPIRE leaderboard:game:daily:2026-03-18 604800` : returns 1 (success)
- `TTL leaderboard:game:daily:2026-03-18` : returns 604791 seconds remaining

6. Mini Case Study – Country-Wise Quiz Leaderboard

```
127.0.0.1:6379> ZADD leaderboard:quiz:global 950 tenzin
(integer) 0
127.0.0.1:6379> ZADD leaderboard:quiz:global 870 dorji
(integer) 0
127.0.0.1:6379> ZADD leaderboard:quiz:global 910 pema
(integer) 0
127.0.0.1:6379> ZADD leaderboard:quiz:global 980 rahul
(integer) 0
127.0.0.1:6379> ZADD leaderboard:quiz:global 930 priya
(integer) 0
127.0.0.1:6379> ZADD leaderboard:quiz:global 860 amit
(integer) 0
127.0.0.1:6379> ZADD leaderboard:quiz:country:BT 950 tenzin
(integer) 0
127.0.0.1:6379> ZADD leaderboard:quiz:country:BT 870 dorji
(integer) 0
127.0.0.1:6379> ZADD leaderboard:quiz:country:BT 910 pema
(integer) 0
127.0.0.1:6379> ZADD leaderboard:quiz:country:IN 980 rahul
(integer) 0
127.0.0.1:6379> ZADD leaderboard:quiz:country:IN 930 priya
(integer) 0
127.0.0.1:6379> ZADD leaderboard:quiz:country:IN 860 amit
(integer) 0
127.0.0.1:6379> ZREVRANGE leaderboard:quiz:global 0 4 WITHSCORES
 1) "rahul"
 2) "980"
 3) "tenzin"
 4) "950"
 5) "priya"
 6) "930"
 7) "pema"
 8) "910"
 9) "dorji"
10) "870"
127.0.0.1:6379> ZREVRANGE leaderboard:quiz:country:BT 0 2 WITHSCORES
 1) "tenzin"
 2) "950"
 3) "pema"
 4) "910"
 5) "dorji"
 6) "870"
127.0.0.1:6379> ZREVRANGE leaderboard:quiz:country:IN 0 2 WITHSCORES
 1) "rahul"
 2) "980"
 3) "priya"
 4) "930"
 5) "amit"
 6) "860"
127.0.0.1:6379> ZREVRANK leaderboard:quiz:global tenzin
(integer) 1
127.0.0.1:6379> ZREVRANK leaderboard:quiz:country:BT tenzin
(integer) 0
```

```

(venv) macbookairm4chip@Tandin-0mu redis-practicals % python quiz_leaderboard.py
Adding quiz scores...

--- Global Top 5 ---
#1 rahul: 980.0
#2 tenzin: 950.0
#3 priya: 930.0
#4 pema: 910.0
#5 dorji: 870.0

--- Bhutan (BT) Top 3 ---
#1 tenzin: 950.0
#2 pema: 910.0
#3 dorji: 870.0

--- India (IN) Top 3 ---
#1 rahul: 980.0
#2 priya: 930.0
#3 amit: 860.0

--- Individual Rankings ---
Tenzin - Global Rank: #2, BT Rank: #1
Rahul - Global Rank: #1, IN Rank: #1
(venv) macbookairm4chip@Tandin-0mu redis-practicals %

```

Key Schema Design

Leaderboard Type	Key Pattern	Example
Global	leaderboard:quiz:global	All players worldwide
Country	leaderboard:quiz:country:{code}	leaderboard:quiz:country:BT

Implementation

A `QuizLeaderboard` class was created to support:

- Adding scores to both the global and country-specific leaderboards at the same time
- Querying top players globally
- Querying top players for specific countries
- Getting individual ranks, both global and country-specific

Sample Data Added

Global Leaderboard:

Rank	Player	Score
1	rahul	980
2	tenzin	950
3	priya	930
4	pema	910
5	dorji	870

Bhutan (BT) Leaderboard:

Rank	Player	Score
1	tenzin	950
2	pema	910
3	dorji	870

India (IN) Leaderboard:

Rank	Player	Score
1	rahul	980
2	priya	930
3	amit	860

Individual Rankings

Player	Global Rank	Country Rank
Tenzin	#2	BT #1
Rahul	#1	IN #1

8. Key Learnings

- The use of sorted sets automatically orders the data according to the score
- ZREVRANK returns ranks as 0-based, hence the need to add 1 before displaying
- The use of ZREVRANGE WITHSCORES to get the top players in descending order
- The efficient use of ZINCRBY to update the score without needing to read first
- The use of proper key naming conventions, such as leaderboard:quiz:country:BT, to manage data
- The use of a class to contain Redis commands to maintain clean code